

01 GPU 开卷速查

一分钟速查

GPU 题通常不是考复杂 CUDA 语法，而是考：索引、有效线程、warp divergence、CUDA 改 CPU、带宽/计算受限。

来源优先级：

- EXE/gpu_exe.pdf：一维 CUDA/SIMT 分析，复习课重点讲过。
- EXE/gpu_exe_sol.pdf：对应答案。
- EXE/EXE2.pdf：Problem 1, 3D mean filter / 三维索引 / gridDim 向上取整。
- EXE/EXE2-sol.pdf：对应答案。

注意：目前复习课和本章只把 EXE/EXE2.pdf Problem 1 纳入 GPU 重点；EXE2 Problem 2 不在当前 GPU 考纲重点里，除非老师后续另行点名。

题型对应来源：

题型	主要来源
线程总数 / 有效线程 / 某线程 global index / 最后一个 block 无效线程 / warp divergence / CUDA 改 CPU / bandwidth-bound	EXE/gpu_exe.pdf
三维索引 / blockDim、gridDim / 三维 bounds check / grid 向上取整	EXE/EXE2.pdf Problem 1

核心句：

CUDA kernel 写的是 “一个 thread 的逻辑”；GPU launch 后很多 threads 并行执行这段逻辑。

基本概念

- Grid：一次 kernel launch 的全部 blocks。
- Block：一组 threads，同一个 block 内线程可共享资源。

- Thread：执行 kernel 的最小逻辑单位。
- Warp：SIMT 执行单位，课件默认 32 threads。
- SM：Streaming Multiprocessor，GPU 的主要计算单元。
- SIMT：Single Instruction Multiple Threads，同一 warp 内共享指令流，但各 thread 有自己的数据；可近似把一个 warp 看成执行上的整体，同一时刻不能真正同时执行两条不同分支，所以若 warp 内线程因条件不同走向 if/else 两边，就会发生 divergence，GPU 需要把两条路径分开执行。

常用内建变量：

变量	含义
blockIdx.x/y/z	当前 block 在 grid 中的位置
threadIdx.x/y/z	当前 thread 在 block 中的位置
blockDim.x/y/z	每个 block 的 thread 维度（block 大小）
gridDim.x/y/z	grid 的 block 维度（grid 内 block 数量，gridDim.x = [width / blockDim.x]

blockDim 和 gridDim 来自 launch configuration：

```
kernel<<<gridDim, blockDim>>>(...);
```

如果写成一维：

```
kernel<<<N, T>>>(...);
```

等价于：

```
gridDim = dim3(N, 1, 1);
blockDim = dim3(T, 1, 1);
```

所以没显式写的 .y/.z 默认是 1。

必背公式

一维全局索引：

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
```

理解：先跳过前面完整 blocks，再加当前 block 内偏移。

三维坐标：

```
int x = blockIdx.x * blockDim.x + threadIdx.x;
int y = blockIdx.y * blockDim.y + threadIdx.y;
int z = blockIdx.z * blockDim.z + threadIdx.z;
```

三维线性化：

```
int idx = z * (width * height) + y * width + x;
```

grid 向上取整：

```
dim3 gridDim((width + blockDim.x - 1) / blockDim.x,
              (height + blockDim.y - 1) / blockDim.y,
              (depth + blockDim.z - 1) / blockDim.z);
```

一维简写：

```
numBlocks = (n + threadsPerBlock - 1) / threadsPerBlock
```

意义：grid 至少覆盖全部数据；多出来的 threads 用 bounds check 挡掉。

题型模板

1. 线程总数 / 有效线程

来源：EXE/gpu_exe.pdf 第 1 题

题面：

```
kernel<<<numBlocks, threadsPerBlock>>>(...);
```

模板：

```
总线程数 = numBlocks * threadsPerBlock
有效线程数 = 满足 idx < n 的线程数，通常是 n
无效线程数 = 总线程数 - 有效线程数
```

例：<<<16, 64>>>, n = 1000

```
总线程数 = 16 * 64 = 1024
有效线程 = idx 0..999, 共 1000
无效线程 = 24
```

开卷提示：先看 launch 创建多少 thread，再看 if (idx < n) 过滤多少 thread。

2. 某个 thread 的 global index

来源：EXE/gpu_exe.pdf 第 2 题

题面常给 $\text{blockIdx.x} = 5, \text{threadIdx.x} = 17, \text{blockDim.x} = 64$ 。

```
idx = blockIdx.x * blockDim.x + threadIdx.x
idx = 5 * 64 + 17 = 337
```

人话：第 5 个 block 前面有 5 整块，每块 64 个；进到当前 block 后再偏移 17。

3. 最后一个 block 哪些线程无效

来源：EXE/gpu_exe.pdf 第 3 题

例： $\langle\langle\langle 16, 64 \rangle\rangle\rangle, n = 1000$

最后一个 block：

```
blockIdx.x = 15
idx = 15 * 64 + threadIdx.x = 960 + threadIdx.x
```

有效条件：

```
960 + threadIdx.x < 1000
threadIdx.x < 40
```

结论：

```
有效: threadIdx.x = 0..39
无效: threadIdx.x = 40..63
```

开卷提示：所有 threads 都被创建了；无效只是没通过 bounds check。

4. Warp divergence

来源：EXE/gpu_exe.pdf 第 4 题、第 5 题

只在同一个 warp 内讨论。warp size 默认 32。

判断顺序：

1. 写出该 warp 覆盖的 idx 范围。
2. 先过外层 bounds check，例如 $\text{idx} < n$ 。
3. 再看内层分支，例如 $\text{idx} \% 2 == 0$ 。
4. 分别数每条分支 active lanes。

例 1：第一个 warp 覆盖 $\text{idx} = 0..31$ 。

```
if (idx % 2 == 0)
    ...
```

```
else
    ...
```

结论：偶数 16 lanes，奇数 16 lanes，发生 divergence。

例 2：最后 block 第二个 warp， $idx = 992..1023, n = 1000$ 。

先过 $idx < 1000$ ：只有 992..999 共 8 lanes active

偶数：992, 994, 996, 998, 共 4

奇数：993, 995, 997, 999, 共 4

其余 24 lanes 被 bounds check 挡掉

易错点：不要一看到 % 2 就说 16/16；最后一个 warp 可能只有部分 lanes alive。

5. CUDA 改 CPU 单线程

来源：EXE/gpu_exe.pdf 第 6 题

CUDA 的意思：每个 GPU thread 处理一个 idx 。

CPU 改写：用 for 循环让一个线程依次处理所有 idx 。

```
void f_cpu(float *x, float *y, float *out, int n) {
    for (int idx = 0; idx < n; idx++) {
        float v = x[idx] + y[idx];
        if (idx % 2 == 0)
            out[idx] = 2.0f * v;
        else
            out[idx] = v + 1.0f;
    }
}
```

开卷提示：不要机械翻译 CUDA 语法；核心是“一个 GPU thread = CPU 循环一次迭代”。

6. Bandwidth-bound / compute-bound

来源：EXE/gpu_exe.pdf 第 7 题

操作强度：

$OI = \text{FLOPs} / \text{bytes}$

常见例子：

读 $x[idx]$ ：4 bytes

读 $y[idx]$ ：4 bytes

写 $out[idx]$ ：4 bytes

总计：12 bytes

计算：约 2 FLOPs

$OI = 2 / 12 = 1/6$ FLOPs/byte

判断句：

读写很多字节，但每个元素只做很少计算，所以通常是 bandwidth-bound，不是 compute-bound。

7. CUDA 程序流程填空

来源：课件套路题；可与 [courseware/review.pdf](#) 一起看（不对应当前已标注 EXE 单题）

标准顺序：

1. CPU 端准备 host data。
2. `cudaMalloc` 分配 device memory。
3. `cudaMemcpy(..., cudaMemcpyHostToDevice)` 拷到 GPU。
4. 设置 `blockDim` 和 `gridDim`。
5. `kernel<<<gridDim, blockDim>>>(...)` 启动 kernel。
6. `cudaDeviceSynchronize()` 等待完成。
7. `cudaMemcpy(..., cudaMemcpyDeviceToHost)` 拷回 CPU。
8. `cudaFree` 释放 device memory。

EXE2：3D 题速解

来源：EXE/EXE2.pdf Problem 1 第 1 问、第 2 问、第 3 问、第 4 问

3D 题不要被 $x/y/z$ 吓住，本质仍然是：

一个 thread 负责一个元素，只是这个元素先用三维坐标 (x, y, z) 表示。

解题流程：

1. 用 `blockIdx * blockDim + threadIdx` 分别算 $x/y/z$ 。
2. 用 bounds check 判断坐标是否合法。
3. 如果数组是一维存储，再线性化为 `idx`。

典型 bounds check：

```
if (x < width && y < height && z < depth) {
    ...
}
```

含义：当前 thread 对应的空间坐标必须真的存在。

向上取整的原因：

```
gridDim.x = ceil(width / blockDim.x)
gridDim.y = ceil(height / blockDim.y)
gridDim.z = ceil(depth / blockDim.z)
```

C/CUDA 写法：

```
(size + block - 1) / block
```

人话：grid 的任务是把数据“包住”；边缘多出来的 threads 靠 bounds check 不工作。

考场检查清单

看到 GPU 题，按这个顺序扫：

1. 一个 thread 负责什么？idx 还是 (x,y,z)？
2. idx / (x,y,z) 怎么由 blockIdx/threadIdx/blockDim 算？
3. launch 一共创建多少 threads / blocks？
4. 哪个 if 在过滤无效 thread？
5. 问 divergence 时：先 bounds check，再数分支。
6. 问 3D 时：分别检查 x/y/z 是否越界。
7. 没写 .y/.z 时，默认是 1。

高频术语

- GPU: Graphics Processing Unit，图形处理器。
- GPGPU: General Purpose GPU，通用 GPU 计算。
- SM: Streaming Multiprocessor，流式多处理器。
- SIMT: Single Instruction Multiple Threads，单指令多线程。
- Warp: 一组 SIMT 执行线程，默认 32 threads。
- Kernel: GPU 上执行的函数。
- Grid / Block / Thread: CUDA 线程层次。
- Device memory: GPU 内存。
- Host memory: CPU 侧内存。
- Thread divergence: 分支发散。
- Memory bandwidth: 内存带宽。
- Operational intensity: 操作强度，FLOPs per byte。